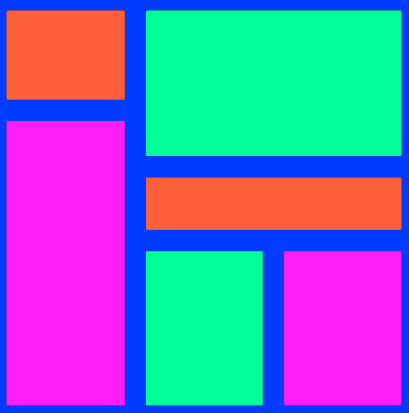
{EPITECH}

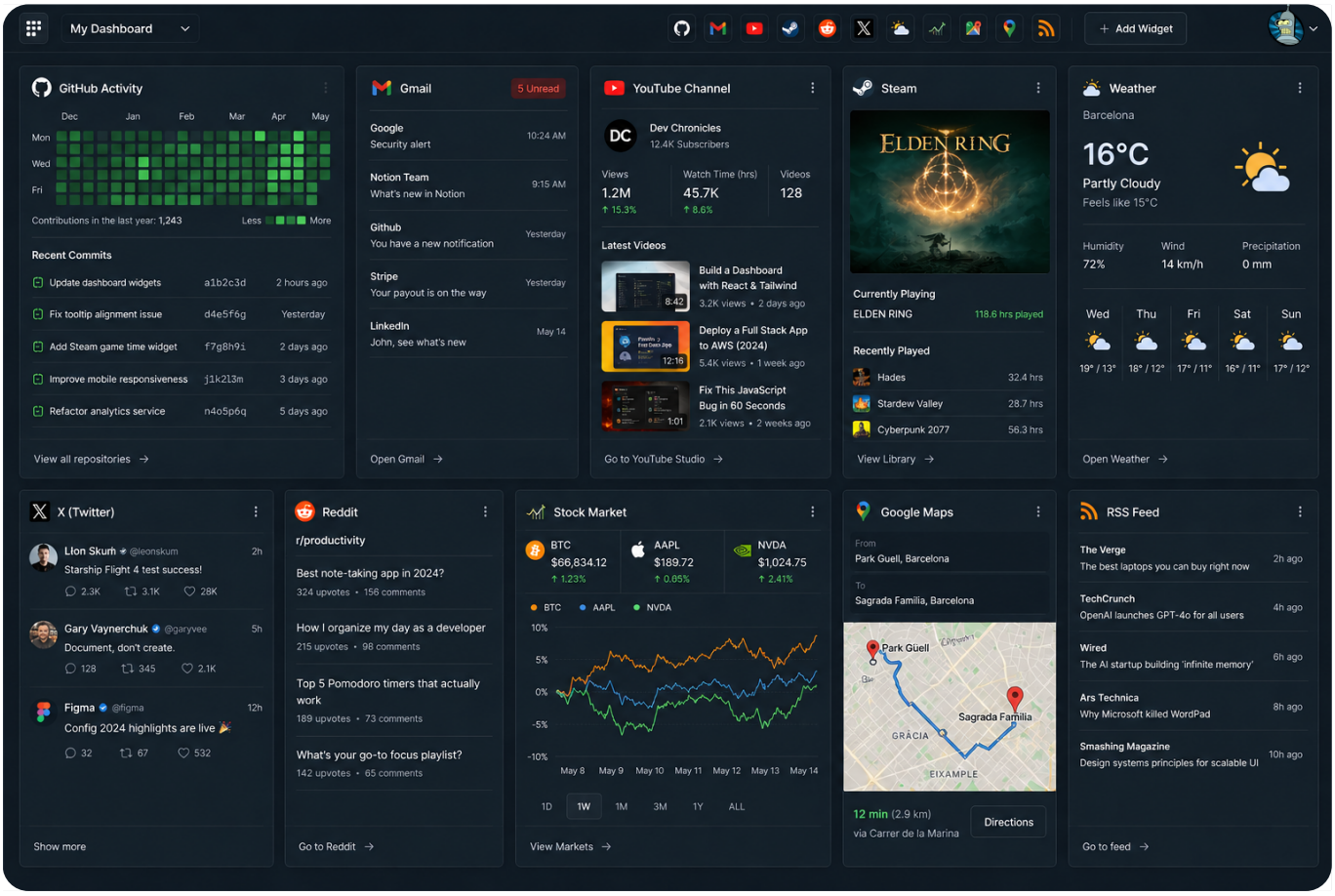
DASHBOARD_

< BUILT BY YOU. FOR YOU. />



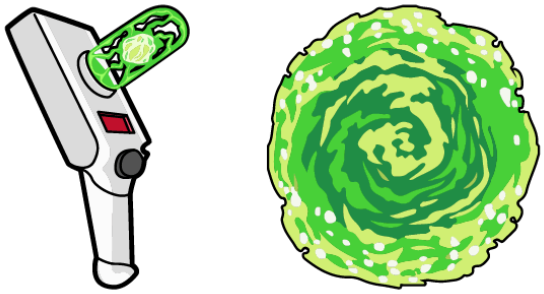
DASHBOARD

Let's familiarize ourselves with full-stack development tools and techniques through the creation of a highly customizable dashboard that gathers information from multiple sources in real time.



The application allows any user — especially you — to build a custom dashboard to monitor, visualize, and analyze any digital content, such as feeds, streams, social networks, analytics, etc.

Its interface could replace the browser's default new-tab page with a customizable control panel, where widgets act like a portal gun opening doors to different digital worlds.



Main components & features

The application must support the following workflow and actions:

- ✓ users first create an account and confirm their **registration** before accessing the platform;
- ✓ once **authenticated**, they can subscribe to **services**;
- ✓ each service provides a set of **widgets**;
- ✓ users can build a personalized **dashboard** by adding configured widget instances;
- ✓ a **timer** mechanism is used to periodically refresh the data displayed by these widgets.



We will evaluate both the quality and quantity of your work. For a group of **X students**, the application must support at least **(1 + X) services** and a total of **(3 × X) widgets**.

User management

Since the application centers on the information displayed to users, it must include a way to manage this information. This requires implementing a user management module that allows unauthenticated users to register through a form, using common approaches such as email signup or third-party authentication.



To manage site users, an administration section would be very useful.

Identity & permissions

Using the application requires two complementary mechanisms:

- ✓ **authentication** identifies the user and binds data to a specific account. This can be basically handled with credentials (username + password) or via OIDC.
- ✓ **authorization** allows your application widgets to access external services and APIs. This must be done via OAuth 2.0.



When implementing OAuth, remember to link the third-party account to an existing platform user.

Services

Authenticated users must first select the services for which they have an account.

To subscribe to a service, users must provide their credentials (e.g., fill in their username and password from their profile page, or link their account via OAuth).

Some services, such as weather, require no external account and are available by default to any authenticated user. Others remain unavailable until the user explicitly subscribes via the application.



Personal data and credentials are highly sensitive. Handle them with extreme care.

Widgets

Each service should offer widgets (with various purpose, appearance, configurations, etc.).
Widget instances can be configured, then added to the dashboard.

For inspiration, find below **a non-exhaustive list of services** and what they could display:

- ✓ Weather: sky overview, temperature, precipitations.
- ✓ Communication platform: unread messages, reactions count per user, last posts from a channel.
- ✓ Streaming services: channel's subscriber count, trending videos, views count, scheduled events.
- ✓ Scholar / professional information: date and location of next appointments, delivery deadlines.
- ✓ GitHub: recent commits messages and authors, security alerts.
- ✓ Steam: most played games for the current month, news feed for a specific game.
- ✓ Monitoring: number of connections to a server, uptime of a service, error rates over time.
- ✓ Market data: exchange rate for a currency pair, price history for one or more assets.
- ✓ Maps: directions to go to the next appointment, roadmap of the next bike ride.
- ✓ Cinema: today's screenings for a given cinema.
- ✓ RSS: last items from a given feed.



To be valid, your widgets must offer some configuration options.
A widget with no configurable parameters is invalid and will not be counted.

Timer

This essential component is responsible for refreshing the information displayed by the widget instances on the dashboard.

To achieve this, each widget instance on the dashboard communicates its refresh rate to the Timer, which triggers the necessary data updates at the appropriate intervals.

Dashboard

The main page of the application is the dashboard.
From this page, authenticated users can:

- ✓ Add a new widget instance, which requires to:
  - Select a widget type;
  - Configure the selected widget;
  - Set the widget's refresh rate;
  - Confirm to add the widget to the dashboard.
- ✓ Reconfigure a widget instance present on the dashboard.
- ✓ Move a widget instance present on the dashboard.
- ✓ Delete a widget instance present on the dashboard.



Two instances of the same widget, each with a different configuration, should be able to display distinct information at the same time.



Deliverables and presentation

Your application provides a **high-quality**, **responsive** interface with good UX and **accessibility** features.

Your repository must include the following deliverables:

- ✓ **Application source code**:
  all source files required to build and run the application (no binaries, temp files, or object files) ;
- ✓ **README.md**:
  concise documentation of your project, aimed to developers and users.
- ✓ **Technology choices**:
  a document, in the README or linked from it, that compares at least 3 options on the same criteria for each layer (frontend, backend and database), then justifies the chosen ones based on your project needs and your team's skills.
- ✓ **Proof of concept**:
  before full development, build a proof of concept that proves your chosen stack works: one configurable widget that fetches real data from an external API and displays it.
  Keep it in a `poc` folder at the root of your repository.

You must respect the constraints provided below.
You are also expected to present and demonstrate your project during a keynote.



Recording a short video showcasing your main features is an effective way to avoid disruptions during the live demonstration.

Documentation expectations

Your documentation should provide a clear and practical entry point to your project.

Any README.md should contain, at least and not necessarily in this order:

- ✓ a list of the key features, services and widgets available,
- ✓ some precise and reproducible steps to install and build the app,
- ✓ an overview of the app purpose and main use cases,
- ✓ some clear instructions to run and use the app,
- ✓ a description of the technologies used and the overall structure of the system.

It could also integrate some design schemes: there is no need to make diagrams of the entire project, but rather to identify and document the parts that are most important to understand.



You MUST always adapt your communication to your audience: end users may have no technical background, developers and contributors may have heterogeneous levels of understanding.

Constraints



The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery. All the bonus files (including a potential specific Makefile) should be in a directory named bonus.

For your application, you are free to use the languages and frameworks of your choice.
However, we recommended the following ones: *Elixir, PHP, Ruby, Python, Node.js, Java, .NET, Kotlin, C++, React, Angular, Vue, Bootstrap, Tailwind*.



You will have to justify your technological choices in a professional manner.

In order to standardize the build process, you MUST use:

✓ **docker-compose build**
Deliver a docker-compose.yml file at the root of your project, describing the different docker services used, and including at least the Docker service used to launch the application.

✓ **docker-compose up**
This request validate the integrity of your application. The server service MUST run on port 8080 and respond to GET requests at http://localhost:8080/about.json with the following properties:

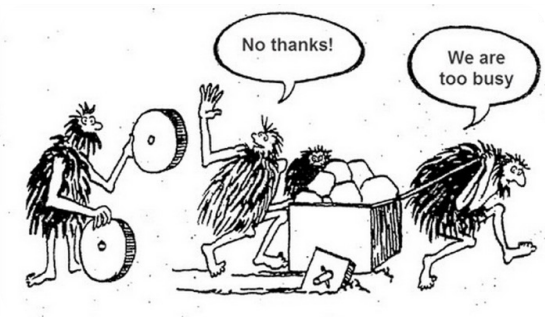
- `client.host`
  indicates the IP address of the client performing the HTTP request
- `server.current_time`
  indicates the server time in the Epoch Unix Time Stamp format
- `server.services`
  indicates the list of services supported by the server
- `server.services[].name`
  indicates the name of the service
- `server.services[].widgets`
  indicates the list of widgets supported by this service
- `server.services[].widgets[].name`
  indicates the identifier of this widget
- `server.services[].widgets[].params`
  indicates the list of parameters to configure this widget
- `server.services[].widgets[].params[].name`
  indicates the identifier of this parameter
- `server.services[].widgets[].params[].type`
  indicates the type of this parameter
  supported types are: integer, string

Below is a basic example of response for a GET request at http://localhost:8080/about.json:


```
{
  "client": {
      "host": "10.101.53.35"
  },
  "server": {
      "current_time": 1531680780,
      "services": [
          {
              "name": "weather",
              "widgets": [
                  {
                      "name": "city_temperature",
                      "description": "Display temperature for a city",
                      "params": [
                        {
                          "name": "city",
                          "type": "string"
                        }
                        ]
                  }
              ]
          },
          {
              "name": "rss",
              "widgets": [
                  {
                      "name": "article_list",
                      "description": "Displaying the list of the last articles",
                      "params": [
                          {
                              "name": "link",
                              "type": "string"
                          },
                          {
                              "name": "number",
                              "type": "integer"
                          }
                      ]
                  }
              ]
          }
      ]
  }
}
```


{EPITECH}

Advices



Software architects main goal is to understand, select, and integrate a wide range of existing components in order to perform various actions **without reinventing the wheel**.

To implement the so-called *business logic*, you need to write the *glue code* that connects these building blocks to achieve a specific goal, without wasting time and effort.

Before diving in without a plan, we strongly suggest that you take the time to:

- ✓ study the available options and their pros and cons, then choose the *state-of-the-art* component that best fits your needs and project constraints;
- ✓ build your *proof of concept* early, to validate that the selected components behave as expected;
- ✓ define a simple, scalable architecture with no code duplication.

While coding and implementing your application, keep the following principles in mind:

- ✓ **Focus on clear, minimal solutions**. Avoid over-engineering and resist the temptation to design for every possible user or edge case.
- ✓ **Prioritize usability and clarity**. A simple, well-designed interface that feels intuitive is more valuable than a complex and confusing application.
- ✓ **Ensure security by design**. Protect sensitive data, validate all inputs, and avoid exposing credentials or internal implementation details.



v 1.1.3

